AI ENGINEERING STACK

Building a Production RAG Pipeline with LangGraph

From raw PDFs to grounded answers: step by step, with code.

AI Coach John

PROITBRIDGE
Strive For Better Future

Follow

Repost

CONCEPT

What is RAG, and why does it matter?

RAG (Retrieval-Augmented Generation) is a technique that grounds an LLM's answers in real, external data instead of relying only on what the model memorized during training.

WHY IT'S IMPORTANT

- ✓ LLMs hallucinate when they don't know something: RAG gives them a source of truth to lean on
- ✓ No need to fine-tune a model every time your data changes
- ✓ You can cite sources, which builds trust
- ✓ Keeps answers current with your latest documents

COMMON USE CASES:

- Internal company docs
- Support product manuals
- Legal & policy Q&A
- Code doc assistants

ARCHITECTURE

Why LangGraph instead of plain LangChain?

LangGraph is a library (built by the LangChain team) for building LLM applications as **stateful graphs** instead of linear chains.

LANGCHAIN (CHAINS)

- ✕ Runs mostly linearly: good for simple pipelines
- ✕ Awkward once you need loops, retries, or branching logic
- ✕ Harder to debug flow hidden inside chain composition

LANGGRAPH (GRAPHS)

- ✓ Models **nod** and **edg** with app as shared state
- ✓ Native support for conditional branching & retries
- ✓ Built-in persistence & human-in-the-loop support
- ✓ Explicit graph makes debugging & visualization easy

WHAT WE NEED BEFORE BUILDING:

Python 3.10+ | langgraph & langchain | sentence-transformers | faiss-cpu | pypdf | Open-source LLM

ROADMAP

What we're building today

An end-to-end RAG pipeline, orchestrated as a LangGraph graph:

1. PDF Loading	2. Chunking	3. Embeddings
4. FAISS Store	5. Retrieval	6. Reranking
7. Prompting	8. Generation	9. Evaluation

Dependencies to install:

```
pip install langgraph langchain langchain-community \
            sentence-transformers faiss-cpu pypdf \
            transformers accelerate ragas
```

- ✓ **pypdf:** PDF extraction
- ✓ **langchain:** text chunking
- ✓ **sentence-transformers:** embed + rerank
- ✓ **faiss-cpu:** vector search
- ✓ **transformers:** open LLM
- ✓ **ragas:** RAG evaluation

STEP 1 OF 9

Extracting text from PDFs

Reads raw PDF files and pulls out clean text so it can be processed downstream. This is the entry point of the pipeline: garbage in, garbage out, so clean extraction matters.

```
from langchain_community.document_loaders import PyPDFLoader

def load_pdf(file_path: str):
    loader = PyPDFLoader(file_path)
    documents = loader.load() # returns 1 Document per page
    return documents

docs = load_pdf("company_handbook.pdf")
print(f"Loaded {len(docs)} pages")
```

KEY TAKEAWAYS

- **Page-by-page wrapping:** `PyPDFLoader` reads the PDF page-by-page and wraps each page in a `Document` object with `page_content` and `metadata`.
- **Why metadata matters:** Keeping metadata here matters: later, when the LLM cites an answer, you want to know which page it came from.
- **Scanned PDFs need OCR:** For scanned/image PDFs you'd add OCR (e.g., `unstructured` or `pytesseract`): plain text extraction won't work there.

STEP 2 OF 9

Splitting documents into chunks

Breaks large documents into smaller, overlapping pieces so each chunk is small enough to embed meaningfully and fit into the LLM's context window later.

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

def chunk_documents(documents, chunk_size=500, chunk_overlap=50):
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=chunk_size, chunk_overlap=chunk_overlap,
        separators=["\n\n", "\n", ". ", " ", ""]
    )
    return splitter.split_documents(documents)

chunks = chunk_documents(docs)
print(f"Created {len(chunks)} chunks")
```

HOW IT WORKS

- **Semanti focus:** `chunk_s` keeps each piece small enough to be semantically focused (too big = diluted embeddings, too small = lost context).
- **Preserving boundaries:** `chunk_over` prevents important info from being cut off exactly at a chunk boundary.
- **Natural boundaries first:** `RecursiveCharact` tries to split on paragraphs and sentences first before falling back to words.

STEP 3 OF 9

Turning text into vectors

Converts each text chunk into a numerical vector that captures its semantic meaning, so we can later compare "how similar" a question is to a chunk mathematically.

```
from sentence_transformers import SentenceTransformer

embedding_model = SentenceTransformer("BAAI/bge-small-en-v1.5")

def embed_chunks(chunks):
    texts = [chunk.page_content for chunk in chunks]
    return embedding_model.encode(
        texts, normalize_embeddings=True, show_progress_bar=True
    )

vectors = embed_chunks(chunks)
```

WHY THESE SETTINGS?

- **Open-source efficiency:** `BAAI/bge-small-en-v1.5` is a strong, free, open-source embedding model: small enough to run on CPU, good enough for most tasks.
- **Reliable cosine search:** `normalize_embed` scales vectors to unit length, making cosine similarity search reliable.
- **Semantic vector space:** Each chunk becomes a fixed-length vector (384 dimensions): similar meanings end up close in vector space.

STEP 4 OF 9

Storing vectors for fast search

Stores all chunk embeddings in FAISS, a library built for extremely fast similarity search over millions of vectors: this becomes our searchable knowledge base.

```
import faiss
import numpy as np

def build_faiss_index(vectors):
    dimension = vectors.shape[1]
    index = faiss.IndexFlatIP(dimension) # inner product = cosine
    index.add(np.array(vectors).astype("float32"))
    return index

faiss_index = build_faiss_index(vectors)
faiss.write_index(faiss_index, "knowledge_base.index")
```

FAISS ADVANTAGES

- **Inner product math:** `IndexF` does an exact inner-product search: since our embeddings are normalized, this equals cosine similarity.
- **Zero API cost:** FAISS runs entirely locally (no API calls, no cost) and is fast enough for most use cases up to millions of vectors.
- **Disk persistence** We persist the index to disk so we don't have to re-embed everything every time we run the app.

STEP 5 OF 9

Finding relevant chunks for a query

Takes the user's question, embeds it the same way, and searches FAISS for the top-k most similar chunks.

```
def retrieve(query: str, index, chunks, top_k=5):
    query_vector = embedding_model.encode(
        [query], normalize_embeddings=True
    ).astype("float32")

    scores, indices = index.search(query_vector, top_k)
    return [chunks[i] for i in indices[0]]

results = retrieve("What is the leave policy?", faiss_index, chunks)
```

RETRIEVAL RULES

- **Same model requirement:** The query must be embedded with the *sa* model used for the chunks: otherwise the vector spaces won't line up.
- **Tuni k:** `top_` controls how many candidate chunks we pull: larger k gives the reranker more to work with but costs more compute downstream.
- **Coarse filter:** This is "coarse" retrieval: fast but not perfectly precise, which is exactly why we rerank next.

STEP 6 OF 9

Reranking for precision

Re-scores the retrieved chunks using a more accurate (but slower) cross-encoder model, so the most relevant chunks end up at the top before we hand them to the LLM.

```
from sentence_transformers import CrossEncoder

reranker = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")

def rerank(query, retrieved_chunks, top_n=3):
    pairs = [(query, c.page_content) for c in retrieved_chunks]
    scores = reranker.predict(pairs)
    ranked = sorted(
        zip(scores, retrieved_chunks), key=lambda x: x[0], reverse=True
    )
    return [chunk for _, chunk in ranked[:top_n]]

top_chunks = rerank("What is the leave policy?", results)
```

WHY RERANK?

- **Cross-encoder precision:** Vector search is fast but approximate; a cross-encoder looks at query and chunk *toge* for higher relevance accuracy.
- **Efficiency balance:** We only rerank the small top-k set from retrieval (not the whole database): this keeps it fast while boosting precision.
- **Reducing noise:** `top_n=` trims down to just the best chunks, reducing noise in the final prompt.

STEP 7 OF 9

Building the prompt

Combines the retrieved context and the user's question into a structured prompt that instructs the LLM to answer only from the provided context.

```
def build_prompt(query, top_chunks):
    context = "\n\n".join([c.page_content for c in top_chunks])
    return f"""Answer using ONLY the context below.
If not in context, say "I don't have enough information."

Context:
{context}

Question: {query}
Answer:"""

final_prompt = build_prompt("What is the leave policy?", top_chunks)
```

PROMPT ENGINEERING TIPS

- **Reducing hallucinations:** Explicitly telling the model to stick to the context is a simple but effective way to reduce hallucination.
- **Safe fallbac** The fallback instruction ("say I don't have enough information") prevents the model from making things up when retrieval comes up empty.
- **Wording consistency:** Keep this prompt clean and consistent: small wording changes can noticeably affect answer quality.

STEP 8 OF 9

Generating the answer

Sends the constructed prompt to an open-source LLM to generate the final, grounded answer.

```
from transformers import pipeline

generator = pipeline(
    "text-generation",
    model="mistralai/Mistral-7B-Instruct-v0.2",
    device_map="auto"
)

def generate_answer(prompt, max_new_tokens=300):
    out = generator(prompt, max_new_tokens=max_new_tokens, do_sample=False)
    return out[0]["generated_text"][len(prompt):].strip()

print(generate_answer(final_prompt))
```

GENERATION SETTINGS

- **Strong instruction model:** `Mistral-7B-Instruct` is a solid open-source choice; runs well on GPU with strong instruction following.
- **Deterministic output:** `do_sample` makes output deterministic: ideal for factual RAG answers without creative variance.
- **Production scaling:** In production, swap for a hosted inference endpoint (HF Endpoints, vLLM, Ollama) for better latency.

ORCHESTRATION

Putting it all together as a LangGraph

Wraps every step into a LangGraph graph: functions become nodes, and the graph controls the flow (supporting retries & conditional branches).

```
from langgraph.graph import StateGraph, END
from typing import TypedDict, List

class RAGState(TypedDict):
    query: str; retrieved: List; reranked: List; prompt: str; answer: str

def retrieve_node(s): s["retrieved"] = retrieve(s["query"], faiss_index, chunks); return s
def rerank_node(s): s["reranked"] = rerank(s["query"], s["retrieved"]); return s
def prompt_node(s): s["prompt"] = build_prompt(s["query"], s["reranked"]); return s
def generate_node(s): s["answer"] = generate_answer(s["prompt"]); return s

graph = StateGraph(RAGState)
graph.add_node("retrieve", retrieve_node); graph.add_node("rerank", rerank_node)
graph.add_node("prompt", prompt_node); graph.add_node("generate", generate_node)
graph.set_entry_point("retrieve"); graph.add_edge("retrieve", "rerank")
graph.add_edge("rerank", "prompt"); graph.add_edge("prompt", "generate")
graph.add_edge("generate", END)

rag_app = graph.compile()
print(rag_app.invoke({"query": "What is the leave policy?"})["answer"])
```

- **Shared State:** `RAGStat` is passed between nodes: every node reads and writes to it.
- **Modular Nodes:** Each step is an explicit node: easy to test or reorder independently.
- **Graph Power:** Easily add conditional edges (e.g., loop back if rerank score is low).

STEP 9 OF 9

Evaluating the RAG pipeline

Measures whether the pipeline is actually retrieving relevant context and generating faithful, accurate answers using automated RAG evaluation metrics.

```
from ragas import evaluate
from ragas.metrics import faithfulness, answer_relevancy, context_precision
from datasets import Dataset

eval_data = Dataset.from_dict({
    "question": ["What is the leave policy?"],
    "answer": [result["answer"]],
    "contexts": [[c.page_content for c in result["reranked"]]],
    "ground_truth": ["Employees get 18 paid leave days per year."]
})

scores = evaluate(eval_data, metrics=[faithfulness, answer_relevancy,
context_precision])
print(scores)
```

RAGAS METRICS

- **faithfulness:** Checks if answer is supported by context (catches hallucination).
- **answer_relevancy:** Checks if answer directly addresses the question asked.
- **context_precision** Checks if retrieved chunks were actually useful and relevant.
- **Data-driven tuning:** Run regularly after changes to measure real pipeline improvements.

CONCLUSION

That's a full RAG pipeline, end to end

All orchestrated as a clean, stateful LangGraph graph. Here is your complete architecture recap:

1. PDF Loading → 2. Chunking → 3. Embeddings

4. FAISS Store → 5. Retrieval → 6. Reranking

7. Prompting → 8. Generation → 9. Evaluation

Save this for later
Follow for more hands-on AI engineering breakdowns.